AI Ops Assignment 5

Madhav Arun Krishna — DA24B012

1 Environment

For this assignment, I ran everything CPU-only on Ubuntu Linux with Python 3.12.3. My laptop has an AMD Ryzen 7 8845HS with 8 physical cores, 16 logical CPUs, and 32 GB RAM. The Part A results use model v1 with one Uvicorn worker.

2 Part A

Below I have given a brief overview of the FastAPI service. It handles the required prediction format, the range checks, the cross-field rule, request IDs, the health endpoint, Prometheus metrics, and generic inference errors. In the video demonstration we can also see the 15 tests pass.

Concurrency	Requests	req/s	p50 (ms)	p95 (ms)	p99 (ms)	Errors
1	1598	399.5	2.4	3.3	3.7	0
2	2253	563.2	3.3	5.3	6.2	0
4	2127	531.8	7.3	10.0	11.3	0
8	2114	528.5	14.8	19.0	21.5	0
16	2073	518.2	30.6	36.0	40.1	0

Table 1: Concurrency sweep results

We see that throughput reaches its peak at concurrency 2 and then stays around 500–530 req/s, while latency keeps increasing. This means that the saturation knee is approximately 2. This makes sense from Little's Law, once throughput becomes flat, adding more concurrent requests mostly means that more requests are waiting. In the extended sweep, concurrency 32 gave 495.8 req/s with p95 77.2 ms, while concurrency 40 gave 501.2 req/s with p95 131.6 ms. Thus, the measured one-worker capacity under the p95 $\leq$ 80 ms SLO is approximately 496 req/s.

At concurrency 1, batch size 1 costs 2.4 ms per row. Batch size 100 takes 2.7 ms for the full request, which is only 0.027 ms per row. This is to be expected because the cost of HTTP, JSON, validation, DataFrame creation, and the model call is now shared by 100 rows. In order to improve capacity, I then used two Uvicorn workers. This gave 932.5 req/s at concurrency 48 with p95 63.0 ms. At concurrency 64, throughput increased to 1090.2 req/s, but p95 also increased to 84.8 ms and failed the SLO. So the measured capacity improved by $932.5/495.8 = 1.88\times$. It is close to double, but not exactly double.

The mean can hide the slow requests because they get averaged with a large number of fast requests. p95 and p99 directly show us what happens to the slowest 5% and 1%, and hence they are more useful for a user-facing SLO and for finding queueing problems.

3 Part B

For this part, I wrapped both bundles in a custom pyfunc that returns eta_minutes and model_version. Each logged model also has an input example, an input/output signature, and pinned requirements. I used the required SQLite-backed registry.

Registry	Bundle	Alias	Evaluation	MAE	Rush MAE	Rain MAE	p95 ms
1	v1	prod	prod	11.955	19.680	11.623	1.09
2	v2	candidate	candidate	6.105	8.580	6.285	1.14

Table 2: Metrics of prod and candidate

The gate uses fresh data from make_data(2000, seed=99) and times 200 single-row predictions. We see that all four conditions pass with the default thresholds, so we get promote=True. The relative MAE improvement is about 48.9%. When I change min_rel_gain to 0.60, only the gain condition fails and we get promote=False. I also wrote nine synthetic tests that check whether the condition checking logic works correctly.

For the shadow test, I send the same 200 fresh rows to prod on port 5101 and candidate on port 5102. The mean absolute difference is 9.484 minutes, and 35.0% of predictions differ by more than 10 minutes. The HTTP p95 values are 5.7 ms for prod and 5.8 ms for candidate. So based on this, I would promote v2. It improves the overall MAE and both slice MAEs by a large amount, while still passing the latency gate. However, the labelled fresh data shows that v2 is usually correcting v1 rather than making it worse. Moving an alias alone does not change a running server, because the alias is resolved when the model gets loaded. We can pick up the new model without dropping requests using a rolling restart. Another possibility is to load the newly resolved version and atomically swap it after the old model finishes its in-flight requests.

4 Part C

Here I used a two-CPU Ray cluster, 1,000 customer IDs, and a canary weight of 0.2. SHA-256 maps every customer ID to one stable bucket. We see that 19.2% of customers reach v2, which is within the allowed range. All 200 customers keep the same arm on a second call, and all 200 still keep it after I redeploy the same application. When I increase the weight to 0.5, no existing v2 customer moves back to v1. Finally, all 50 requests with X-Beta: 1 reach v2.

Python's built-in hash() is salted separately for each process. Thus the same customer could get a different assignment after a restart or in another replica. SHA-256 does not have this problem and stays the same across processes and machines. Stickiness is important because a customer should not keep seeing different ETA behaviour. It is also important for the experiment, because customers moving between groups would make the comparison unreliable.

I chose a minimum of one candidate replica and a maximum of three. This keeps the idle cost small but still gives the canary space to handle a burst. A target of one ongoing request and a two-second upscale delay lets Ray react before a large queue builds up. The ten-second downscale delay avoids repeatedly removing replicas during short gaps. Note

that two prod replicas and three candidate replicas at 0.25 CPU each, along with the 0.1-CPU router, reserve only 1.35 CPUs and hence fit on the two-CPU cluster.

5 Part D

Lastly, the smoke test passes eight checks and exits with code 0. It checks health, the model version on health and inference, a valid prediction, two model-sanity relationships, invalid input returning 422, and p95 $\leq$ 80 ms. During a 10% v2 canary, I would abort if any of the following happens:

1. v2 HTTP 5xx rate $>$ 1% over five minutes, after at least 100 requests;
2. v2 request-latency p95 $>$ 80 ms over five minutes;
3. v2 MAE more than 5% worse than v1 over one hour, after at least 100 labelled deliveries;
4. observed v2 traffic share outside 8–12% over ten minutes, after at least 500 requests.

6 Not Tested List

All the above numbers come from actually running the code. Relevant screenshots can be found in the folders and I have also shown as much as possible in the video demonstration.

7 AI Use Disclosure

Can be found in AI_DISCLOSURE.md